Lesson 8 Worksheet: Implementing the Navigator Class — ANSWER KEY

Part A: Navigator Class Design

1. Three attributes the Navigator needs:

Attribute	Type	Purpose
self.position	tuple	Stores the robot's current (row, col) on the grid
self.heading	int	Stores the heading as a number: 0=N, 1=E, 2=S, 3=W
self.line_track	LineTrack	Controls physical robot movement using line tracking from Module 2

2. Three main methods:

Method Name	Input	Output / Action
get_needed_heading	next_pos (tuple)	Returns the heading number (0-3) to reach the next position
turn_to	needed_heading (int)	Turns robot with right turns using clockwise counting
drive_path	path (list of tuples)	Drives the robot through every position in the path

3. Why does the Navigator store its own position instead of getting it from Manhattan?

The Navigator tracks its physical position after each step as it drives. Manhattan only plans paths – it doesn't know where the robot actually is after driving. The Navigator needs its own position to calculate which heading to face for the next step.

4. What object does the Navigator use to physically move the robot?

A LineTrack object, created by calling `LineTrack()` in `__init__`. The LineTrack class (from Module 2) provides `turn_right()` for turning and `track_until_cross()` for driving from one intersection to the next.

Part B: get_needed_heading() Exercises

Navigator Position	Next Position	row_diff	col_diff	Heading
(0, 0)	(1, 0)	1 - 0 = 1	0 - 0 = 0	2 (S)
(2, 1)	(1, 1)	1 - 2 = -1	1 - 1 = 0	0 (N)
(1, 2)	(1, 3)	0	1	1 (E)
(3, 3)	(3, 2)	0	-1	3 (W)
(0, 1)	(1, 1)	1	0	2 (S)
(2, 0)	(2, 1)	0	1	1 (E)
(3, 2)	(2, 2)	-1	0	0 (N)
(1, 3)	(1, 2)	0	-1	3 (W)

Part C: turn_to() Exercises

Current Heading	Needed Heading	Clockwise Steps	Right Turns	New Heading
0 (N)	1 (E)	0->1 = 1	1	1 (E)
0 (N)	2 (S)	0->1->2 = 2	2	2 (S)
1 (E)	1 (E)	already there = 0	0	1 (E)
1 (E)	0 (N)	1->2->3->0 = 3	3	0 (N)
2 (S)	3 (W)	2->3 = 1	1	3 (W)
2 (S)	0 (N)	2->3->0 = 2	2	0 (N)
3 (W)	2 (S)	3->0->1->2 = 3	3	2 (S)
3 (W)	1 (E)	3->0->1 = 2	2	1 (E)
0 (N)	3 (W)	0->1->2->3 = 3	3	3 (W)
1 (E)	3 (W)	1->2->3 = 2	2	3 (W)

- How many required 0 right turns? 1 (E to E)
- How many required 2 right turns (a 180)? 4 (N to S, S to N, W to E, E to W)

Part D: drive_path() Tracing

First path: `[(1, 0), (2, 0), (2, 1), (2, 2)]`

Starting position: (0, 0), Starting heading: 0 (N)

Step (i)	next_pos	Needed Heading	Current Heading	Right Turns	New Heading	New Position
1	(1, 0)	2 (S)	0 (N)	count 0->1->2 = 2	2 (S)	(1, 0)
2	(2, 0)	2 (S)	2 (S)	already there = 0	2 (S)	(2, 0)
3	(2, 1)	1 (E)	2 (S)	count 2->3->0->1 = 3	1 (E)	(2, 1)
4	(2, 2)	1 (E)	1 (E)	already there = 0	1 (E)	(2, 2)

Final position: **(2, 2)**, Final heading: **1 (E)**

Note on step 2: already facing the right way so `straight(8)` is called to clear the intersection before `track_until_cross()`. Same for step 4.

Second path: `[(3, 2), (3, 1), (2, 1), (1, 1), (0, 1)]`

Starting position: (3, 3), Starting heading: 1 (E)

Step (i)	next_pos	Needed Heading	Current Heading	Right Turns	New Heading	New Position
1	(3, 2)	3 (W)	1 (E)	count 1->2->3 = 2	3 (W)	(3, 2)
2	(3, 1)	3 (W)	3 (W)	already there = 0	3 (W)	(3, 1)
3	(2, 1)	0 (N)	3 (W)	count 3->0 = 1	0 (N)	(2, 1)
4	(1, 1)	0 (N)	0 (N)	already there = 0	0 (N)	(1, 1)
5	(0, 1)	0 (N)	0 (N)	already there = 0	0 (N)	(0, 1)

Final position: **(0, 1)**, Final heading: **0 (N)**

Note on steps 2, 4, 5: already facing the right way so `straight(8)` *is called to clear the intersection before* `track_until_cross()`.

Part E: Code Completion

```
def __init__(self, start, heading):
    self.position = start
    self.heading = heading
    self.line_track = LineTrack()
```

Blanks: start, heading, LineTrack()

```
def get_needed_heading(self, next_pos):
    row_diff = next_pos[0] - self.position[0]
    col_diff = next_pos[1] - self.position[1]
    if row_diff == -1:
        return 0
    elif col_diff == 1:
        return 1
    elif row_diff == 1:
        return 2
    elif col_diff == -1:
        return 3
```

Blanks: 0, 0, 1, 1, 0, 1, 2, 3

```
def turn_to(self, needed_heading):
    while self.heading != needed_heading:
        self.line_track.turn_right()
        self.heading = self.heading + 1
        if self.heading == 4:
            self.heading = 0
```

Blanks: needed_heading, turn_right, 1, 4, 0

```
def drive_path(self, path):
    for next_pos in path:
        needed = self.get_needed_heading(next_pos)
        if self.heading == needed:
            self.line_track.drivetrain.straight(8)
        self.turn_to(needed)
```

```
        self.line_track.track_until_cross()
        self.position = next_pos
```

Blanks: path, next_pos, needed, 8, needed, next_pos

Why does `drive_path()` call `self.line_track.drivetrain.straight(8)` when the robot is already facing the right way?

When the robot doesn't need to turn, it is still sitting on the previous intersection's crossing line. The `straight(8)` call drives 8 cm forward to clear the intersection so that `track_until_cross()` can look for the NEXT crossing line without immediately re-detecting the current one.

Why does the loop iterate directly with `for next_pos in path:` instead of skipping elements?

Because the path does not include the starting position – every element is a new cell the robot needs to drive to. The Navigator already knows its current position through `self.position`, so the path only contains the destinations to visit.

Reflection

The Navigator class reuses your LineTrack class from Module 2. Why is this a good design choice?

Reusing LineTrack means the Navigator doesn't need to implement line tracking, crossing detection, or turning from scratch. The LineTrack class already handles following a line (`track_until_cross()`) and making precise turns (`turn_right()`). This is composition – the Navigator "has a" LineTrack and delegates physical movement to it. If you improve LineTrack (like better PID tuning), every class that uses it benefits automatically. It also means each class has a single clear job: LineTrack handles low-level sensor-based driving, and Navigator handles high-level path navigation.